

 Open in Colab

In [6]: `!pip install -q pylatexenc`

```

 0.0/162.6 kB ? eta -:--:--
162.6/162.6 kB 6.0 MB/s eta 0:00:00
Preparing metadata (setup.py) ... done
Building wheel for pylatexenc (setup.py) ... done
```

In [1]: `!pip install qiskit qiskit-aer matplotlib scipy qiskit-ibm-runtime`

Collecting qiskit

Downloading qiskit-2.5.0-cp310-abi3-manylinux_2_28_x86_64.whl.metadata (14 kB)

Collecting qiskit-aer

Downloading qiskit_aer-0.17.2-cp312-cp312-

manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (8.3 kB)

Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)

Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (1.16

Collecting qiskit-ibm-runtime

Downloading qiskit_ibm_runtime-0.48.0-py3-none-any.whl.metadata (23 kB)

Requirement already satisfied: numpy<3,>=2.0 in /usr/local/lib/python3.12/dist-packages (from qiskit) (2.0.2)

Collecting rustworkx>=0.15.0 (from qiskit)

Downloading rustworkx-0.18.0-cp310-abi3-

manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (10 kB)

Requirement already satisfied: dill>=0.3 in /usr/local/lib/python3.12/dist-packages (from qiskit) (0.3.8)

Collecting stevedore>=3.0.0 (from qiskit)

Downloading stevedore-5.9.0-py3-none-any.whl.metadata (2.3 kB)

Requirement already satisfied: typing-extensions in /usr/local/lib/python3.12/dist-packages (from qiskit) (4.16.0)

Requirement already satisfied: psutil>=5 in /usr/local/lib/python3.12/dist-packages (from qiskit-aer) (5.9.5)

Requirement already satisfied: python-dateutil>=2.8.0 in /usr/local/lib/python3.12/dist-packages (from qiskit-aer) (2.9.0.post0)

Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)

Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)

Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.63.0)

Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.5.0)

Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (26.2)

Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)

Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)

Requirement already satisfied: requests>=2.32.4 in /usr/local/lib/python3.12/dist-packages (from qiskit-ibm-runtime) (2.32.4)

Collecting requests-ntlm>=1.1.0 (from qiskit-ibm-runtime)

Downloading requests_ntlm-1.3.0-py3-none-any.whl.metadata (2.4 kB)

Requirement already satisfied: urllib3>=2.4.0 in /usr/local/lib/python3.12/dist-packages (from qiskit-ibm-runtime) (2.5.0)

Collecting ibm-platform-services>=0.61.1 (from qiskit-ibm-runtime)

Downloading ibm_platform_services-0.77.0-py3-none-any.whl.metadata (9.3 kB)

Collecting ibm-quantum-schemas<1,>=0.9.20260612 (from qiskit-ibm-runtime)

Downloading ibm_quantum_schemas-0.9.20260612-py3-none-any.whl.metadata (3.0 kB)

Requirement already satisfied: pydantic>=2.11.0 in /usr/local/lib/python3.12/dist-packages (from qiskit-ibm-runtime) (2.13.4)

Collecting pybase64>=1.4.0 (from qiskit-ibm-runtime)

Downloading pybase64-1.4.3-cp312-cp312-

manylinux1_x86_64.manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_5_x86_64.whl.metadata (8.7 kB)

Collecting samplomatic>=0.18.0 (from qiskit-ibm-runtime)

Downloading samplomatic-0.20.0-py3-none-any.whl.metadata (6.5 kB)

Collecting ibm_cloud_sdk_core<4.0.0,>=3.24.4 (from ibm-platform-services>=0.61.1->qiskit-ibm-runtime)

Downloading ibm_cloud_sdk_core-3.26.0-py3-none-any.whl.metadata (8.7 kB)

```

Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from pydantic>=2.11.0->qiskit-ibm-runtime) (0.7.0)
Requirement already satisfied: pydantic-core==2.46.4 in /usr/local/lib/python3.12/dist-packages (from pydantic>=2.11.0->qiskit-ibm-runtime) (2.46.4)
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from pydantic>=2.11.0->qiskit-ibm-runtime) (0.4.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.0->qiskit-aer) (1.17.0)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.4->qiskit-ibm-runtime) (3.4.9)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.4->qiskit-ibm-runtime) (3.18)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.4->qiskit-ibm-runtime) (2026.6.17)
Requirement already satisfied: cryptography>=1.3 in /usr/local/lib/python3.12/dist-packages (from requests-ntlm>=1.1.0->qiskit-ibm-runtime) (49.0.0)
Collecting pypng>=0.4.0 (from requests-ntlm>=1.1.0->qiskit-ibm-runtime)
  Downloading pypng-0.12.1-py3-none-any.whl.metadata (4.1 kB)
Requirement already satisfied: orjson>=3.9.0 in /usr/local/lib/python3.12/dist-packages (from samplomatic>=0.18.0->qiskit-ibm-runtime) (3.11.9)
Requirement already satisfied: cffi>=2.0.0 in /usr/local/lib/python3.12/dist-packages (from cryptography>=1.3->requests-ntlm>=1.1.0->qiskit-ibm-runtime) (2.1.0)
Requirement already satisfied: PyJWT<3.0.0,>=2.11.0 in /usr/local/lib/python3.12/dist-packages (from ibm_cloud_sdk_core<4.0.0,>=3.24.4->ibm-platform-services>=0.61.1->qiskit-ibm-runtime) (2.13.0)
Requirement already satisfied: pycparser in /usr/local/lib/python3.12/dist-packages (from cffi>=2.0.0->cryptography>=1.3->requests-ntlm>=1.1.0->qiskit-ibm-runtime) (3.0)
Downloading qiskit-2.5.0-cp310-abi3-manylinux_2_28_x86_64.whl (9.8 MB)
_____ 9.8/9.8 MB 89.2 MB/s eta 0:00:00
Downloading qiskit_aer-0.17.2-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (12.4 MB)
_____ 12.4/12.4 MB 101.3 MB/s eta 0:00:00
Downloading qiskit_ibm_runtime-0.48.0-py3-none-any.whl (2.0 MB)
_____ 2.0/2.0 MB 89.5 MB/s eta 0:00:00
Downloading ibm_platform_services-0.77.0-py3-none-any.whl (412 kB)
_____ 412.6/412.6 kB 32.3 MB/s eta 0:00:00
Downloading ibm_quantum_schemas-0.9.20260612-py3-none-any.whl (111 kB)
_____ 111.3/111.3 kB 11.5 MB/s eta 0:00:00
Downloading pybase64-1.4.3-cp312-cp312-manylinux1_x86_64.manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_5_x86_64.whl (71.6 kB)
_____ 71.6/71.6 kB 7.5 MB/s eta 0:00:00
Downloading requests_ntlm-1.3.0-py3-none-any.whl (6.6 kB)
Downloading rustworkx-0.18.0-cp310-abi3-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (2.4 MB)
_____ 2.4/2.4 MB 95.6 MB/s eta 0:00:00
Downloading samplomatic-0.20.0-py3-none-any.whl (224 kB)
_____ 224.2/224.2 kB 19.6 MB/s eta 0:00:00
Downloading stevedore-5.9.0-py3-none-any.whl (54 kB)
_____ 54.5/54.5 kB 5.2 MB/s eta 0:00:00
Downloading ibm_cloud_sdk_core-3.26.0-py3-none-any.whl (76 kB)
_____ 76.6/76.6 kB 7.2 MB/s eta 0:00:00
Downloading pypng-0.12.1-py3-none-any.whl (130 kB)
_____ 130.2/130.2 kB 9.9 MB/s eta 0:00:00
Installing collected packages: stevedore, rustworkx, pybase64, qiskit, ibm_cloud_sdk_core, samplomatic, qiskit-aer, pypng, ibm-platform-services, requests-ntlm, ibm-quantum-schemas, qiskit-ibm-runtime
Successfully installed ibm-platform-services-0.77.0 ibm-quantum-schemas-0.9.20260612 ibm_cloud_sdk_core-3.26.0 pybase64-1.4.3 pypng-0.12.1 qiskit-2.5.0 qiskit-aer-0.17.2

```

qiskit-ibm-runtime-0.48.0 requests-ntlm-1.3.0 rustworkx-0.18.0 samplomatic-0.20.0
stevedore-5.9.0

```
In [3]: import qiskit
from qiskit_aer import AerSimulator

print("="*50)
print(" EASY: INSTALLATION & BACKEND VERIFICATION ")
print("="*50)

# Verify Qiskit version
print(f"✅ Qiskit Version: {qiskit.__version__}")

# Initialize Local Aer simulator
sim = AerSimulator()
print(f"✅ Aer Simulator Initialized: {sim.name}")
print(f"Available Aer Methods: {sim.available_methods()[:4]}")

=====
EASY: INSTALLATION & BACKEND VERIFICATION
=====
✅ Qiskit Version: 2.5.0
✅ Aer Simulator Initialized: aer_simulator
Available Aer Methods: ('automatic', 'statevector', 'density_matrix', 'stabilizer')
```

```
In [11]: import matplotlib.pyplot as plt
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram

# 1. Create circuit
qc = QuantumCircuit(1, 1)
qc.h(0)
qc.measure(0, 0)

# 2. Simulate
sim = AerSimulator()
compiled_qc = transpile(qc, sim)
counts = sim.run(compiled_qc, shots=1024).result().get_counts()

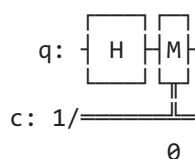
print("1-Qubit Measurement Counts (1024 shots):", counts)

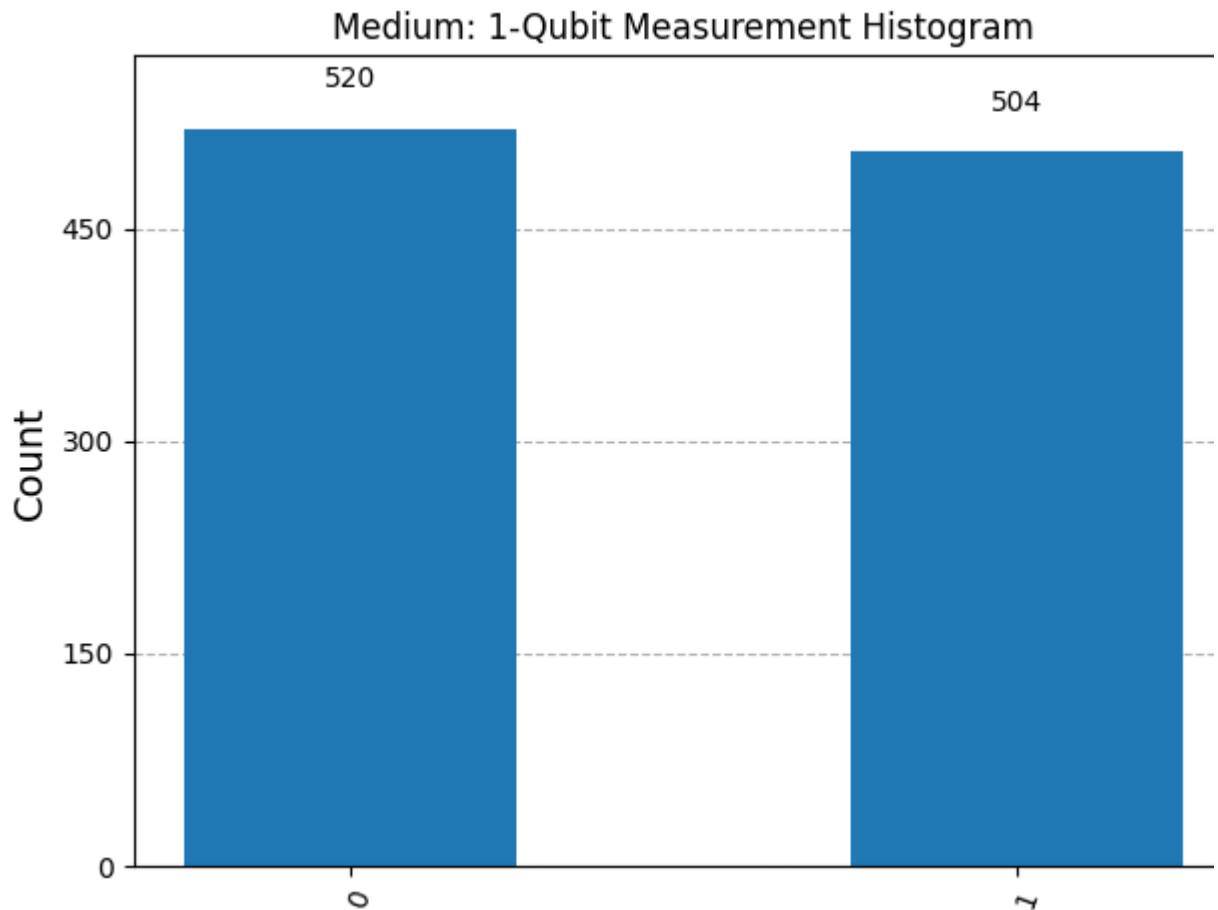
# 3. Graphical Output 1: Text Circuit Diagram (Works instantly without pylatex)
print("\nCircuit Diagram:")
print(qc.draw())

# 4. Graphical Output 2: Histogram (Uses Matplotlib natively)
display(plot_histogram(counts, title="Medium: 1-Qubit Measurement Histogram"))
```

1-Qubit Measurement Counts (1024 shots): {'0': 520, '1': 504}

Circuit Diagram:





```
In [12]: import matplotlib.pyplot as plt
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram

# 1. Create a 3-qubit, 3-classical bit circuit
n_qubits = 3
qc3 = QuantumCircuit(n_qubits, n_qubits)

# Apply Hadamard gate to all 3 qubits
for q in range(n_qubits):
    qc3.h(q)

qc3.measure(range(n_qubits), range(n_qubits))

# 2. Simulate with 8192 shots for statistical precision
shots = 8192
sim = AerSimulator()
compiled_qc3 = transpile(qc3, sim)
counts3 = sim.run(compiled_qc3, shots=shots).result().get_counts()

# 3. Print Probability Breakdown
print(f"🎯 Theoretical Probability per state: {1/8:.4f} (1/8)\n")
print("State Empirical Probabilities:")
for state in sorted(counts3.keys()):
    prob = counts3[state] / shots
    print(f"  |{state}>: {prob:.4f} (Count: {counts3[state]})")

# 4. Circuit Diagram (Pure Text - Always works without dependencies)
print("\nCircuit Diagram:")
print(qc3.draw())
```

```
# 5. Graphical Output: Uniform Probability Histogram
```

```
display(plot_histogram(counts3, title="Hard: 3-Qubit Superposition (Uniform 1/8
```

🔗 Theoretical Probability per state: 0.1250 (1/8)

State Empirical Probabilities:

|000>: 0.1279 (Count: 1048)

|001>: 0.1298 (Count: 1063)

|010>: 0.1246 (Count: 1021)

|011>: 0.1223 (Count: 1002)

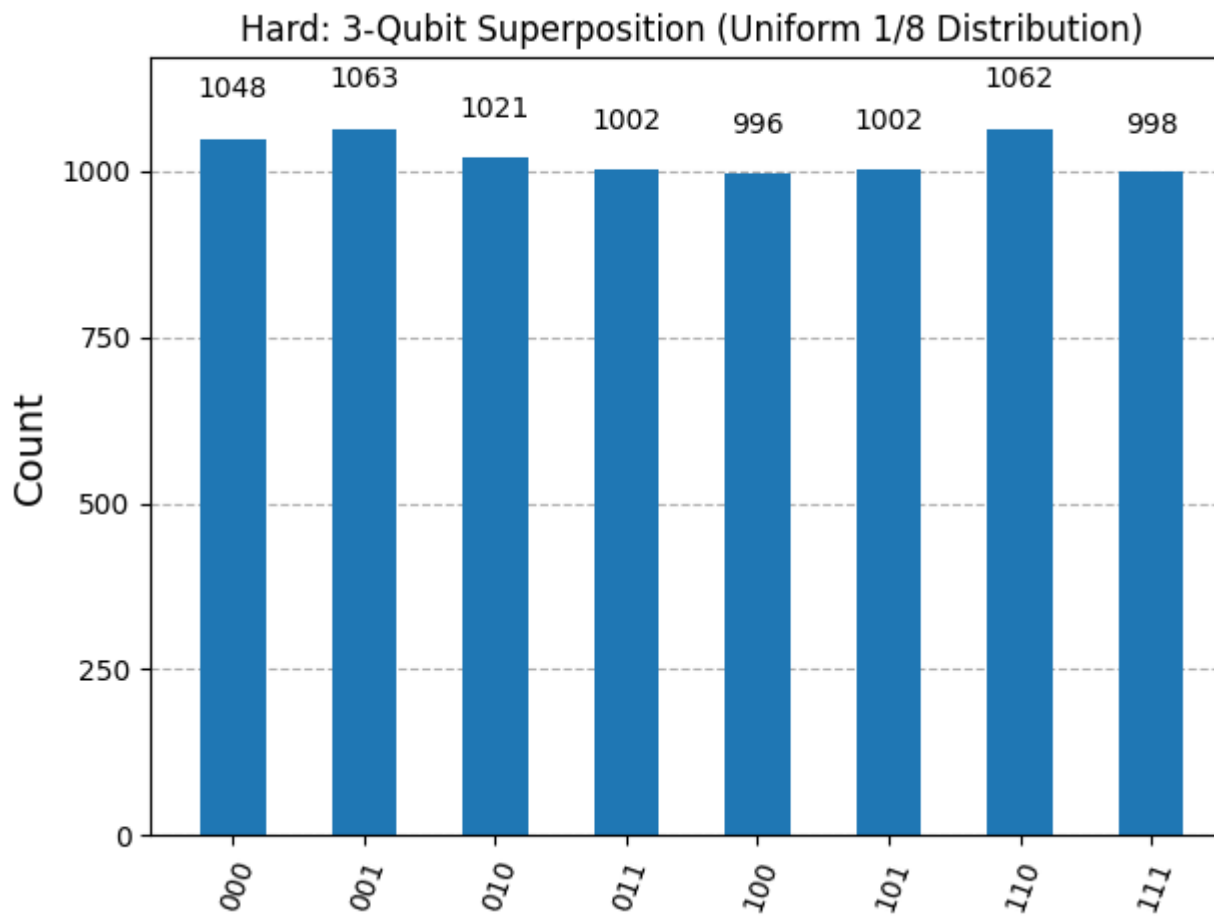
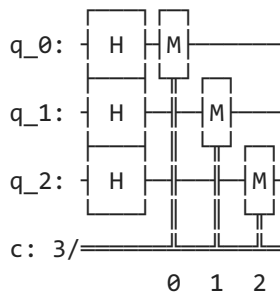
|100>: 0.1216 (Count: 996)

|101>: 0.1223 (Count: 1002)

|110>: 0.1296 (Count: 1062)

|111>: 0.1218 (Count: 998)

Circuit Diagram:



```
In [13]: import random
import matplotlib.pyplot as plt
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator
```

```

# 1. Initialize Aer Simulator
sim = AerSimulator()

def generate_quantum_bits(length=1000):
    """Generates quantum random bits using Hadamard superposition collapse."""
    qc = QuantumCircuit(1, 1)
    qc.h(0)          # Create superposition: (|0> + |1>) / sqrt(2)
    qc.measure(0, 0) # Collapse state upon measurement

    t_qc = transpile(qc, sim)
    counts = sim.run(t_qc, shots=length).result().get_counts()

    # Extract bit outcomes
    zeros = [0] * counts.get('0', 0)
    ones = [1] * counts.get('1', 0)
    bits = zeros + ones
    random.shuffle(bits)
    return bits

# 2. Generate 1000 bits from both sources
N = 1000
qrng_bits = generate_quantum_bits(N)
prng_bits = [random.choice([0, 1]) for _ in range(N)]

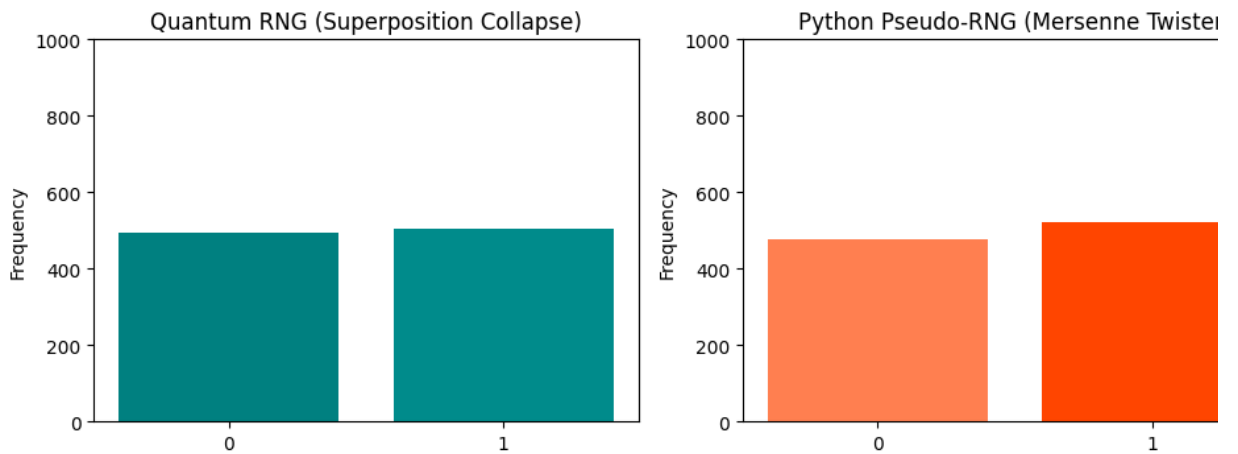
q_ones, p_ones = sum(qrng_bits), sum(prng_bits)

# 3. Print Statistical Output
print(f"

```

 Statistical Summary (1000 bits generated):
 QRNG -> Ones: 506 (50.6%) | Zeros: 494 (49.4%)
 PRNG -> Ones: 523 (52.3%) | Zeros: 477 (47.7%)

Real-World: Quantum vs Pseudorandom Generator Comparison



```
In [14]: import matplotlib.pyplot as plt
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator
from qiskit_aer.noise import NoiseModel, depolarizing_error
from qiskit.visualization import plot_histogram

# 1. Create a 3-qubit GHZ state circuit ( $|000\rangle + |111\rangle$ ) /  $\sqrt{2}$ 
qc = QuantumCircuit(3, 3)
qc.h(0)
qc.cx(0, 1)
qc.cx(0, 2)
qc.measure([0, 1, 2], [0, 1, 2])

shots = 2048

# 2. Run Ideal Simulation
ideal_sim = AerSimulator()
t_qc_ideal = transpile(qc, ideal_sim)
ideal_counts = ideal_sim.run(t_qc_ideal, shots=shots).result().get_counts()

# 3. Build a Hardware-like Noise Model (Depolarizing gate errors)
noise_model = NoiseModel()
error_1q = depolarizing_error(0.02, 1) # 2% error on 1-qubit gates (H gate)
error_2q = depolarizing_error(0.06, 2) # 6% error on 2-qubit gates (CX gates)
noise_model.add_all_qubit_quantum_error(error_1q, ['h'])
noise_model.add_all_qubit_quantum_error(error_2q, ['cx'])

# Run Noisy Simulation
noisy_sim = AerSimulator(noise_model=noise_model)
t_qc_noisy = transpile(qc, noisy_sim)
noisy_counts = noisy_sim.run(t_qc_noisy, shots=shots).result().get_counts()

# 4. Print Results
print("Ideal Simulator Counts (Only  $|000\rangle$  and  $|111\rangle$ ):")
print(" ", ideal_counts)
print("\nNoisy Hardware-like Simulator Counts (Includes error states like  $|001\rangle$ ,")
print(" ", noisy_counts)

# 5. Graphical Output 1: Text Circuit Diagram
print("\nGHZ Entanglement Circuit Diagram:")
print(qc.draw())

# 6. Graphical Output 2: Comparative Histogram Plot
```

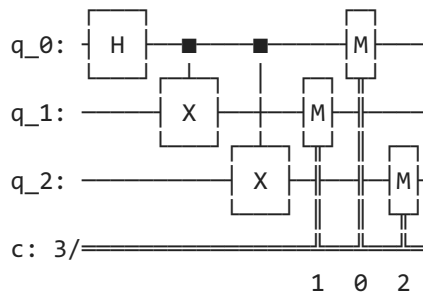


```
display(plot_histogram(
    [ideal_counts, noisy_counts],
    legend=["Ideal Simulator", "Noisy Simulator (Hardware-like)"],
    title="Challenge: Ideal vs Noisy Quantum Hardware Execution"
))
```

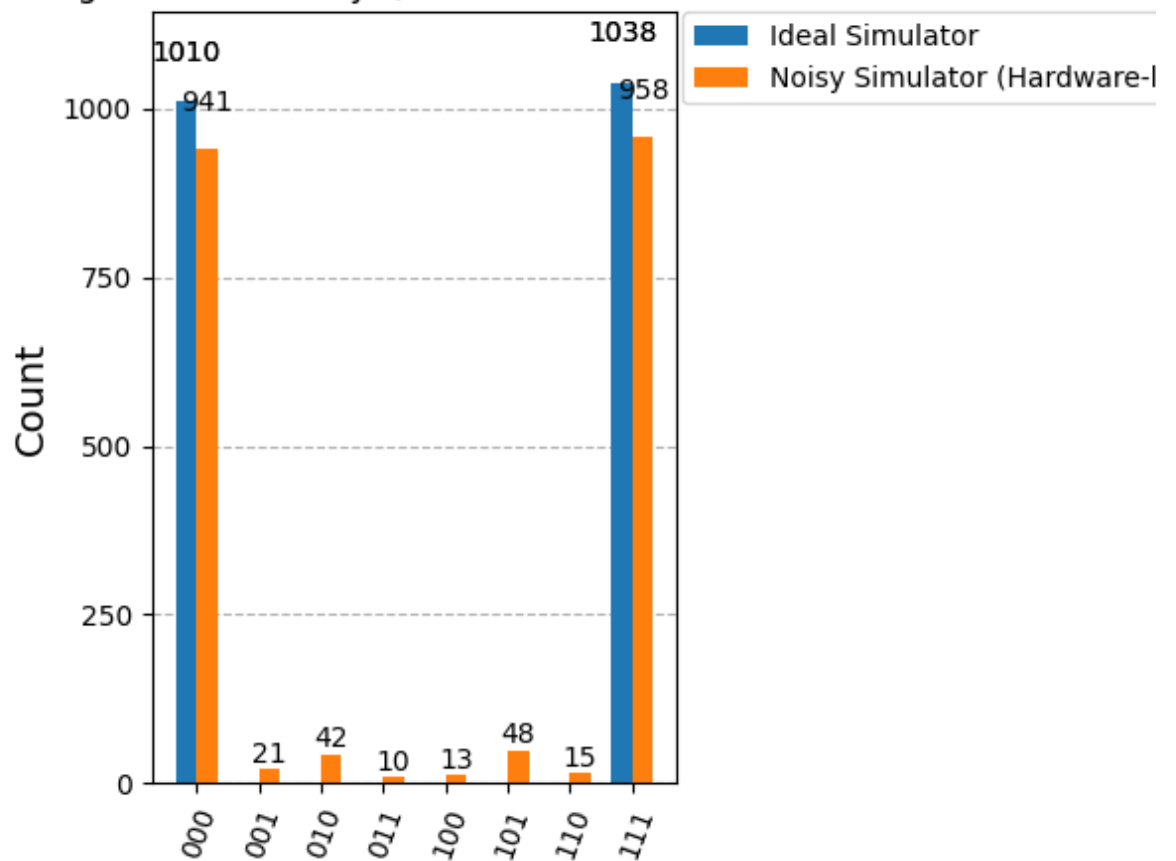
Ideal Simulator Counts (Only $|000\rangle$ and $|111\rangle$):
 {'111': 1038, '000': 1010}

Noisy Hardware-like Simulator Counts (Includes error states like $|001\rangle$, $|010\rangle$, etc.):
 {'110': 15, '100': 13, '101': 48, '001': 21, '010': 42, '000': 941, '011': 10, '111': 958}

GHZ Entanglement Circuit Diagram:



Challenge: Ideal vs Noisy Quantum Hardware Execution



In []: